

OpenEMMA: Open-Source Multimodal Autonomous Driving Framework

Hotha Kaarthikeya – B230662EC

Bhukya Jagadeesh – B230252EC

Pranav D Pandit – B231174EC

Poondla Latheesh – B231170EC

Bommareddy Navaneet Seshi Reddy – B230261EC

Department of Electronics and Communication Engineering
NIT CALICUT

September 21, 2026

Introduction

Modular Driving Stacks Compound Errors

- Perception $\rightarrow$ prediction $\rightarrow$ planning, linked by hand-designed interfaces
- Each interface can lose or wrongly discretize information
- Modules are trained separately, so errors accumulate instead of being optimized toward safe driving

End-to-End MLLMs: A Different Path

- Pretrained on internet-scale data: broad world knowledge and reasoning
- Can map sensor input directly to a trajectory, with no hand-designed interfaces
- **Last semester:** we worked on an end-to-end model (OpenEMMA) using Chain-of-Thought reasoning

Problem Statement

OpenEMMA Limitations

- **Slow:** ≈ 90 s per frame
- **Fragile:** numbers parsed from free text; may fail or return < 10 points
- **Not adapted:** no driving-data training

Accuracy is around (ADE 1.53–2.07 m); cost and reliability are serious issue.

What a Better Solution Needs

- **Fast:** one forward pass instead of several CoT generation passes
- **Reliable:** fixed-size numeric output, no text parsing
- **Adaptive:** trainable on driving data
- **Lightweight:** fine-tunable on academic compute

Objective: OpenEMMA-TinyVLA

One trainable end-to-end model that outputs a fixed-size trajectory in a single forward pass, with no text generation or parsing: a 1.3B LLaVA-Pythia backbone with LoRA and a diffusion policy head (TinyVLA recipe).

Literature Review: EMMA (Waymo)

End-to-End Formulation

Maps multi-sensor input directly to trajectory:

$$O_{\text{trajectory}} = G(T_{\text{intent}}, T_{\text{ego}}, V)$$

- **Inputs:** 6-camera video (V), BEV history (T_{ego}), intent (T_{intent}).
- **Supervision:** Self-supervised on future positions; fine-tuned end-to-end on Gemini.

4-Stage CoT Rationale

Articulates planning steps before waypoints:

- 1 Coarse scene context
- 2 Critical objects (3D/BEV)
- 3 Actor behavior & future motion
- 4 High-level driving decision

Limitation & Takeaway for Our Work

Requires closed foundation models and proprietary datasets (WOMD). However, it validates that **unified vision-to-trajectory modeling** outperforms modular systems.

Literature Review: OpenEMMA

Zero-Shot Inference Strategy

- **Open MLLMs:** Prompts LLaVA-7B, Qwen2.5-VL on nuScenes without training.
- **Lightweight Inputs:** Front camera (with YOLO boxes) + 5 s speed/curvature history.
- **Self-Inferred Intent:** Generates intent via CoT to replace missing navigation inputs.

Kinematic Bicycle Model

Predicts (s, κ) pairs instead of raw (x, y) :

$$\theta_t = \theta_0 + \sum \kappa_i s_i \Delta t, \quad x_t = x_0 + \sum s_i \cos \theta_i \Delta t$$

- Guarantees kinematically feasible, smooth paths with no teleportation.

Bottlenecks (Observed in Our LLaVA-7B & Qwen2.5-VL Runs)

- **Latency:** Autoregressive text generation takes ≈ 90 s per frame across CoT steps.
- **Fragility:** Regex extraction from free text frequently drops points or fails to parse.

Literature Review: TinyVLA & Our Synthesis

TinyVLA Formulation (Robotics)

- **Compact Backbone:** Small LLaVA-Pythia (70M–1.4B) replaces bulky 7B+ LLMs.
- **Diffusion Head:** Continuous action denoising replaces autoregressive text tokens.
- **LoRA Adaptation:** Adapts $\approx 5\%$ backbone weights; trains policy head in full.
- $\approx 20\times$ lower latency than OpenVLA.

How We Unify the Three Works

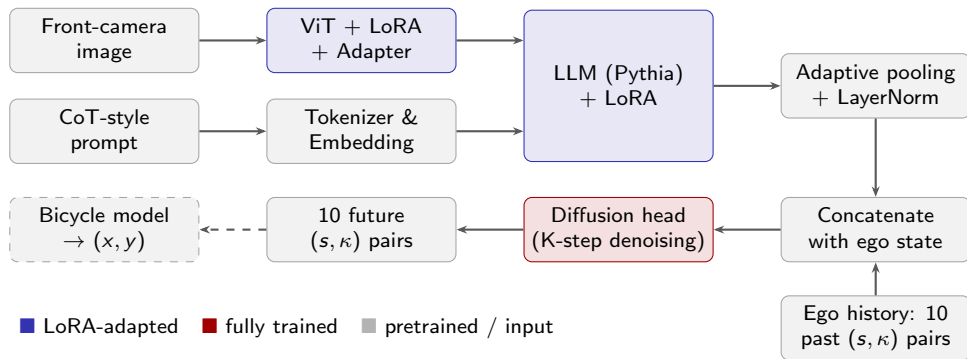
- **EMMA:** Unified end-to-end vision-to-trajectory planning paradigm.
- **OpenEMMA:** 5 s nuScenes horizon + kinematic bicycle model $(s, \kappa) \rightarrow (x, y)$.
- **TinyVLA:** 1D diffusion head on LLaVA-Pythia (1.3B) with LoRA.

Core Benefit: OpenEMMA-TinyVLA

One forward pass + 10-step DDPM denoising: outputs numeric trajectories directly in ≈ 1 s/frame ($\approx 90\times$ faster) with zero text parsing or schema failures.

Methodology

LLaVA-Pythia (1.3B) with LoRA; its text head is replaced by a diffusion policy head: image + prompt + ego history $\rightarrow$ 10 future (speed s , curvature κ) pairs in one pass.



Methodology: LoRA & Diffusion Head

LoRA: Cheap Fine-Tuning

- Freeze the pretrained ViT + LLM weights entirely
- Add small trainable adapter matrices to attention/MLP layers
- Only the adapters are updated $\Rightarrow$ fast, low-cost fine-tuning

$$W' = W + \Delta W, \quad \Delta W = BA, \quad B \in \mathbb{R}^{d \times r}, \quad A \in \mathbb{R}^{r \times k}, \quad r \ll d, k$$

Diffusion Head: Replaces Text Generation

- Instead of generating text, directly predicts continuous trajectory values
- Start from Gaussian noise and denoise it for 10 steps
- Each step is conditioned on the image + prompt representation
- Output: 10 future (speed, curvature) values

$$x_T \sim \mathcal{N}(0, I) \xrightarrow[\text{image} + \text{prompt}]{10 \text{ denoising steps}} x_0 = [v_1, \kappa_1, \dots, v_{10}, \kappa_{10}]$$

Methodology: Training, Inference & Conditioning

The diffusion head learns to turn random noise into the true future trajectory $a_0 \in \mathbb{R}^{10 \times 2}$ (10 future (v, κ) pairs), then reverses that process at inference time.

Training: Corrupt, Then Recover

$$a_j = \underbrace{\sqrt{\bar{\alpha}_j}}_{\text{signal kept}} a_0 + \underbrace{\sqrt{1 - \bar{\alpha}_j}}_{\text{noise added}} \varepsilon$$

- Pick a random step j (1 of $N_{\text{train}}=100$); this fixes how much noise to add to a_0
- The head F_ϕ guesses the noise $\hat{\varepsilon}$; training pushes $\hat{\varepsilon}$ toward the true ε

Inference: Repeat the Reverse

$$\underbrace{a_{j_N} \sim \mathcal{N}(0, I)}_{\text{pure noise}} \xrightarrow{\hat{\varepsilon}_j = F_\phi(a_j, j, c)} \underbrace{a_{j-}}_{\text{less noisy}}$$

- Start from pure noise; at each step, guess the noise and use it to move to a slightly cleaner version
- Repeat $N_{\text{infer}}=10$ times, deterministically ($\eta=0$) $\rightarrow$ final a_0

Conditioning: What Tells F_ϕ What to Do

$$c = \left[\underbrace{\tau(j)}_{\text{noise level}} ; \underbrace{c_H}_{\text{scene + motion}} \right] \quad \hat{h} = \gamma(c) \odot h + \beta(c)$$

- c_H : pooled image + prompt features combined with the ego vehicle's recent motion; $\tau(j)$: an embedding of which noise level j we're at
- h : a feature map inside the U-Net; γ, β , computed from c , scale/shift h at every block

Methodology: Dataset & Training

Dataset: nuScenes

- Front-camera image + ego speed/curvature only
- No lidar, radar, HD maps, or 3D boxes
- 10 past steps (5 s) $\rightarrow$ predict 10 future steps (5 s)

Training Setup

- LoRA fine-tuning: rank 64, $\alpha = 256$
- Diffusion head trained fully (from scratch)
- 5 epochs, small batch size

Train/Test Split

- **Fine-tuning:** 150 scenes from nuScenes
- **Evaluation:** nuScenes-mini (10 unseen scenes)

Results

OpenEMMA-TinyVLA: per-scene ADE (m)

Scene	@1s	@2s	@3s	avg
scene-0061	2.955	4.851	6.693	4.833
scene-0103	2.771	4.641	6.591	4.668
scene-0553	0.339	0.480	0.604	0.474
scene-0655	5.507	9.208	12.946	9.220
scene-0757	0.640	1.025	1.349	1.005
scene-0796	8.478	14.170	19.942	14.196
scene-0916	3.270	5.541	7.815	5.542
scene-1077	9.551	16.119	22.810	16.160
scene-1094	4.215	7.451	11.001	7.556
scene-1100	0.359	0.582	0.785	0.575
Mean	3.809	6.407	9.054	6.423

Protocol: ADE = mean ℓ_2 position error over $T = 2, 4, 6$ keyframes (1, 2, 3 s); avg = mean of the three.

OpenEMMA Baselines (CoT, no fine-tuning)

Model	@1s	@2s	@3s	avg
LLaVA-7B	1.90	1.82	2.50	2.07
Qwen2.5-VL-3B	0.87	1.52	2.19	1.53

Speed vs. Accuracy

- Latency per frame: ≈ 90 s (CoT) vs. ≈ 1 s (ours), about two orders of magnitude faster
- Mean ADE 6.42 m, above both baselines;
- Closest to baselines on simple scenes, e.g. scene-0553 (0.47 m)

Results

Likely Causes of the Accuracy Gap

- **Short training:** only 5 epochs for a diffusion head
- **From-scratch head:** no driving prior, unlike the pretrained backbone
- **Small data:** a 150-scene slice of nuScenes

Next Steps

- Train for substantially more epochs
- Fine-tune on the more training data
- Rebalance learning rates: slower for LoRA, higher for the diffusion head

Conclusion

One model, no free-text generation or parsing, $\sim 90\times$ faster; the accuracy gap is attributed mainly to under-training rather than the approach itself.

References I

-  Hwang, J.-J., Xu, R., Lin, H., Hung, W.-C., Ji, J., Choi, K., Huang, D., He, T., Covington, P., Sapp, B., Zhou, Y., Guo, J., Anguelov, D., and Tan, M. (2025). Emma: End-to-end multimodal model for autonomous driving.
-  Xing, S., Qian, C., Wang, Y., Hua, H., Tian, K., Zhou, Y., and Tu, Z. (2025). Openemma: Open-source multimodal model for end-to-end autonomous driving.
-  Wen, J., Zhu, Y., Li, J., Zhu, M., Tang, Z., Wu, K., Xu, Z., Liu, N., Cheng, R., Shen, C., Peng, Y., Feng, F., and Tang, J. (2025). Tinyvla: Towards fast, data-efficient vision-language-action models for robotic manipulation.